

Gradient Boosting

15 April 2026 23:14

Gradient Boosting is an ensemble learning technique where we build models sequentially, and each new model tries to correct the mistakes of the previous ones.

Instead of building one strong model, we build many weak learners (usually decision trees) and combine them.

Gradient

- Comes from Gradient Descent
- We minimize error using gradients (derivatives)

Boosting

- Means sequential learning
- Each new model focuses on errors of previous models

Gradient Boosting = Boosting using Gradient Descent to minimize error.

In gradient Boosting

We don't directly predict y .

We build a model like this:

$$F(x) = F_0(x) + F_1(x) + F_2(x) + \dots + F_m(x)$$

Where:

- F_0 = initial prediction (simple)
- Each F_m = new model correcting previous error

Step 1: Start with a base model

$$F_0(x) = \arg \min_c \sum L(y_i, c)$$

Find a constant value c that minimizes the total loss

Important:

- $F_0(x)$ is not a model depending on x
- It is just a single constant prediction for all data points

$\arg \min_c$

- Means: "Find the value of c that minimizes

$$\sum_{i=1}^n L(y_i, c)$$

- Total loss between actual values y_i and constant prediction c

Let's Take a Dummy Dataset

Problem: Regression

x	y
1	10
2	20
3	30
4	40

Find $F_0(x)$

We assume:

Predict same value c for all rows

Case 1: Use MSE Loss

Loss:

$$L = \sum_{i=1}^n (y_i - c)^2$$

Try different values of c

♦ If $c = 10$

Loss =

$$(10 - 10)^2 + (20 - 10)^2 + (30 - 10)^2 + (40 - 10)^2 \\ = 0 + 100 + 400 + 900 = 1400$$

♦ If $c = 20$

Loss =

$$(10 - 20)^2 + (20 - 20)^2 + (30 - 20)^2 + (40 - 20)^2 \\ = 100 + 0 + 100 + 400 = 600$$

♦ If $c = 25$

Loss =

$$(10 - 25)^2 + (20 - 25)^2 + (30 - 25)^2 + (40 - 25)^2 \\ = 225 + 25 + 25 + 225 = 500$$

♦ If $c = 30$

Loss =

$$(10 - 30)^2 + (20 - 30)^2 + (30 - 30)^2 + (40 - 30)^2 \\ = 400 + 100 + 0 + 100 = 600$$

Minimum Loss at: $c = 25$

$$F_0(x) = 25$$

This is simply the mean of y values

Important Result

For MSE Loss:

$$F_O(x) = \text{mean}(y)$$

What if Different Loss?

MAE (Absolute Error)

$$L = \sum_{i=1}^n |y_i - c|$$

Then:

$$F_O(x) = \text{median}(y)$$

Log Loss (Classification)

$$F_O(x) \text{ becomes: } \log\left(\frac{p}{1-p}\right)$$

(where p = probability of class 1)

Hence, before learning patterns:

Start with the *best constant guess*

Then:

- Next models improve it step-by-step

Step 2: Compute residuals (errors) / pseudo-residual

$$r_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

Negative gradient of the loss w.r.t prediction (It is a **general formula for the direction to reduce error (loss)**) or This is the **gradient** of the loss function or pseudo-residual

- how loss changes when prediction changes
- Residual is universal, but boosting uses gradient—which changes with loss.

Why Residual = Gradient?

MSE Loss (Regression)

$$L = \sum_{i=1}^n (y_i - F(x_i))^2$$

derivative w.r.t prediction $F(x_i)$

$$\frac{\partial L}{\partial F(x_i)} = -2(y_i - F(x_i))$$

$$-\frac{\partial L}{\partial F(x_i)} = 2(y_i - F(x_i))$$

Negative Gradient $\propto (y_i - F(x_i))$

Which is:

Residual = Actual – Prediction

or

$$-\frac{\partial L}{\partial F(x_i)} \propto (y_i - F(x_i))$$

Residual = same direction

So:

Instead of computing gradients explicitly, we can just use **residuals**

Step 3: Fit a model to residuals (errors become the target (labels) for the next model)

Train a weak learner $h_m(x)$ on residuals

Step 4: Update model

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where:

- η = learning rate

Step-by-Step Working

Let's say we are predicting house prices

Step 1:

- Predict mean price = 50 lakhs

Step 2:

- Calculate errors:
 - Actual: 60 $\rightarrow$ error = +10
 - Actual: 40 $\rightarrow$ error = -10

Step 3:

- Train tree on these errors

Step 4:

- Update prediction:
 - New prediction = old + correction

Step 5:

- Repeat

Each tree:

- Focuses on what previous trees got wrong

Build Gradient Boosting Step-by-Step

We'll use your dataset:

x	y
1	10
2	20
3	30
4	40

Initialize Model

We already computed:

$$F_0(x) = 25$$

STEP 1: Compute Residuals

$$r_i = y_i - F_0(x)$$

x	y	Prediction (25)	Residual
1	10	25	-15
2	20	25	-5
3	30	25	+5
4	40	25	+15

STEP 2: Train First Tree on Residuals

We train a small tree $h_1(x)$ to predict:

Target = residuals

Example simple split:

- If $x \leq 2 \rightarrow$ predict -10
- If $x > 2 \rightarrow$ predict +10

DT computes them as mean of residuals in each region, It is learning:

For small $x \rightarrow$ errors are negative $\rightarrow$ reduce prediction

For large $x \rightarrow$ errors are positive $\rightarrow$ increase prediction

STEP 3: Update Model

$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$

Let's take:

- $\eta = 0.1$

Updated Predictions:

x	Old (25)	Tree Output	New Prediction
---	----------	-------------	----------------

1	25	-10	$25 - 1 = 24$
2	25	-10	24
3	25	+10	26
4	25	+10	26

STEP 4: Compute New Residuals

x	y	New Prediction	Residual
1	10	24	-14
2	20	24	-4
3	30	26	+4
4	40	26	+14

STEP 5: Train Second Tree

Train $h_2(x)$ on new residuals

Example:

- $x \leq 2 \rightarrow -9$
- $x > 2 \rightarrow +9$

◊ STEP 6: Update Again

$$F_2(x) = F_1(x) + 0.1 \cdot h_2(x)$$

New Predictions:

x	Old	Update	New
1	24	-0.9	23.1
2	24	-0.9	23.1
3	26	+0.9	26.9
4	26	+0.9	26.9

So

- Predictions move closer to actual values
- Residuals keep shrinking
- Each tree:
Learns "what is still wrong"

FINAL MODEL

$$F(x) = F_0 + \eta h_1 + \eta h_2$$

Final Formula

$$F(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x)$$

Gradient Boosting = Iteratively fit models on **negative gradients (residuals)** and add them to improve predictions

- Step 1: Start with guess (mean)
- Step 2: Find error
- Step 3: Learn error
- Step 4: Correct prediction
- Repeat

Gradient Boosting works with **any differentiable loss function**:

Problem	Loss Function
---------	---------------

Regression	MSE
------------	-----

Classification	Log Loss
----------------	----------

Why Use Gradient Boosting?

- Handles complex patterns
- Works for regression & classification
- High accuracy (often top performer)
- Works with mixed data types

Challenges

- Overfitting (if too many trees)
- Slow training (sequential)
- Sensitive to hyperparameters

Hyperparameters

Parameter	Meaning
-----------	---------

n_estimators	number of trees
--------------	-----------------

learning_rate	step size
---------------	-----------

max_depth	tree depth
-----------	------------

Real-Life Applications

Finance

- Credit scoring
- Fraud detection

E-commerce

- Recommendation systems

Healthcare

- *Disease prediction*

Industry

- *Demand forecasting*

Why Gradient Boosting if AdaBoost Exists?

AdaBoost:

- *works well with classification*

Gradient Boosting:

- *Works with any loss*
- *More flexible*
- *More powerful*
- *Handles regression + classification*